Team Note of BunCua Team

MinhhHoangPersia - Pain - Lingg

Compiled on September 5, 2026

Contents

1 Template

2 Data Structure

2.1 Segment Tree Walk

```
/* Find first element in range [u, v] have value >= val */
int findPos(int id, int l, int r, int u, int v, ll val) {
  if(l > v || r < u) return n + 1;
  if(st[id] < val) return n + 1;
  if(l == r) return l;
  push(id);
  int mid = (l + r) / 2;
  int res = findPos(id * 2, l, mid, u, v, val);
  if(res == n + 1) {
    res = findPos(id * 2 + 1, mid + 1, r, u, v, val);
  }
  return res;
}
```

2.2 Persistent Segment Tree

```
struct node {
  node *l, *r;
  int v;

  node() {
    l = r = nullptr;
    v = 0;
  }
};

const int MAX_VAL = 300005;
node *t[MAX_VAL];

void refine(node *cur) {
  cur->v = cur->l->v + cur->r->v;
}

void build(node *cur, int l, int r) {
  cur->l = new node();
  cur->r = new node();
  if (l == r) {
    return;
  }
  int mid = (l + r) / 2;
  build(cur->l, l, mid);
  build(cur->r, mid + 1, r);
  refine(cur);
}

void update(node *cur, int l, int r, int u, int x) {
  if (l == r) {
    cur->v += x;
    return;
  }
  int mid = (l + r) / 2;
  if (u <= mid) {
    node *old = cur->l;
    cur->l = new node();
    cur->l->l = old->l;
    cur->l->r = old->r;
    cur->l->v = old->v;
    update(cur->l, l, mid, u, x);
  } else {
    node *old = cur->r;
    cur->r = new node();
    cur->r->l = old->l;
    cur->r->r = old->r;
    cur->r->v = old->v;
    update(cur->r, mid + 1, r, u, x);
  }
  refine(cur);
}

node* update_version(node *prev_root, int l, int r, int pos,
int val) {
  node *cur = new node();
  cur->l = prev_root->l;
  cur->r = prev_root->r;
  cur->v = prev_root->v;
  update(cur, l, r, pos, val);
  return cur;
}

int query(node *cur, int l, int r, int u, int v) {
  if (l > v || r < u) return 0;
  if (l >= u && r <= v) return cur->v;
  int mid = (l + r) / 2;
  return query(cur->l, l, mid, u, v) + query(cur->r, mid + 1,
  r, u, v);
}

int query_diff(node *rootR, node *rootL, int l, int r, int u,
int v) {
  if (l > v || r < u) return 0;
  if (l >= u && r <= v) return rootR->v - rootL->v;
  int mid = (l + r) / 2;
  return query_diff(rootR->l, rootL->l, l, mid, u, v)
       + query_diff(rootR->r, rootL->r, mid + 1, r, u, v);
}
```

2.3 Trie

```
const int LOG_MAX = 29;

struct Node {
  Node* c[2];
  int cnt;

  Node() {
    c[0] = c[1] = nullptr;
    cnt = 0;
  }
```

```
};

void insert(Node* root, int val) {
  Node* cur = root;
  for (int i = LOG_MAX; i >= 0; i--) {
    int x = (val >> i) & 1;
    if (cur->c[x] == nullptr) {
      cur->c[x] = new Node();
    }
    cur = cur->c[x];
    cur->cnt++;
  }
}

bool search(Node* root, int val) {
  Node* cur = root;
  for (int i = LOG_MAX; i >= 0; i--) {
    int x = (val >> i) & 1;
    if (cur->c[x] == nullptr || cur->c[x]->cnt == 0) {
      return false;
    }
    cur = cur->c[x];
  }
  return cur->cnt > 0;
}

int count_occurences(Node* root, int val) {
  Node* cur = root;
  for (int i = LOG_MAX; i >= 0; i--) {
    int x = (val >> i) & 1;
    if (cur->c[x] == nullptr || cur->c[x]->cnt == 0) return 0;
    cur = cur->c[x];
  }
  return cur->cnt;
}

void erase(Node* root, int val) {
  if (!search(root, val)) return;
  Node* cur = root;
  for (int i = LOG_MAX; i >= 0; i--) {
    int x = (val >> i) & 1;
    cur = cur->c[x];
    cur->cnt--;
  }
}

void update(Node* root, int old_val, int new_val) {
  if (!search(root, old_val)) return;
  erase(root, old_val);
  insert(root, new_val);
}
```

3 Graphs

3.1 2 SAT

```
/*
  - For a boolean variable x, we create two nodes: x (1..n)
  and NOT(x) (n+1..2n).
  - A clause (x OR y) is equivalent to: (!x => y) and (!y =>
  x).
  - If x and NOT(x) are in the same Strongly Connected
  Component (SCC),
    it's a contradiction -> NO solution.
  - We assign TRUE to x if its SCC is topologically sorted
  AFTER NOT(x)'s SCC.
    (Tarjan assigns SCC IDs in reverse topological order, so
    col[x] < col[NOT(x)])
*/

const int N = 2e4 + 5;

int n;
vector<int> G[2 * N];
int num[2 * N], low[2 * N], cnt;
int col[2 * N], ct;
vector<int> stk;
int ans[N];

int NOT(int x) {
  return (x <= n ? x + n : x - n);
}

void addedge(int x, int y) {
  G[NOT(x)].push_back(y);
  G[NOT(y)].push_back(x);
}

void tarjan(int u) {
  num[u] = low[u] = ++cnt;
  stk.push_back(u);
  for (int v : G[u]) {
    if (!col[v]) {
      if (num[v]) {
        low[u] = min(low[u], num[v]);
      } else {
        tarjan(v);
        low[u] = min(low[u], low[v]);
      }
    }
  }
  if (num[u] == low[u]) {
    ct++;
    while (true) {
      int v = stk.back();
      stk.pop_back();
      col[v] = ct;
      if (v == u) break;
    }
  }
}

bool solve() {
  rep(i, 1, 2 * n) {
    if (!num[i]) tarjan(i);
  }
  rep(i, 1, n) {
    if (col[i] == col[NOT(i)]) return false;
    ans[i] = (col[i] < col[NOT(i)]);
  }
  return true;
}
```

3.2 LCA O(1)

```
const int N = 1e5 + 5;
const int LOG = 18;

vector<int> G[N];
int tin[N], cnt, rn[N];
int euler[2 * N], ct, first[N];
int depth[N];
int rmq[LOG][2 * N];

void dfs(int u, int p = 0) {
  tin[u] = ++cnt;
  rn[cnt] = u;
  euler[++ct] = tin[u];
  first[tin[u]] = ct;
  for (int v : G[u]) {
    if (v == p) continue;
    depth[v] = depth[u] + 1;
    dfs(v, u);
    euler[++ct] = tin[u];
  }
}

void build_lca() {
  for (int i = 1; i <= ct; i++) {
    rmq[0][i] = euler[i];
  }
  for (int j = 1; j < LOG; j++) {
    for (int i = 1; i <= ct - (1 << (j - 1)); i++) {
      rmq[j][i] = min(rmq[j - 1][i], rmq[j - 1][i + (1 << (j -
      1))]);
    }
  }
}

int lca(int x, int y) {
  x = first[tin[x]];
  y = first[tin[y]];
```

```
  if (x > y) swap(x, y);
  int k = 31 - __builtin_clz(y - x + 1);
  int min_tin = min(rmq[k][x], rmq[k][y - (1 << k) + 1]);
  return rn[min_tin];
}
```